

LQR vs Reinforcement Learning on a Discrete-Time Double Integrator

Ojas Phadake

Personal Portfolio Project | June 16, 2026

Abstract

We compare Linear Quadratic Regulation (LQR) and Proximal Policy Optimisation (PPO) on a discrete-time double integrator, a canonical second-order system in classical control theory. LQR solves the infinite-horizon quadratic cost problem analytically by computing the solution to the Discrete Algebraic Riccati Equation (DARE), yielding an optimal linear feedback law $u_k = -Kx_k$. PPO, a model-free reinforcement learning algorithm, is given only the reward signal $r_k = -(x_k^\top Qx_k + u_k^\top Ru_k)$ and no knowledge of the system dynamics. After 10^5 environment interactions, PPO achieves a total cost of 365.05, compared to the LQR cost of 364.09 — a suboptimality gap of +0.26%. The PPO control signal correlates with the LQR signal at $r = 0.987$, indicating that the learned policy closely approximates the linear feedback law $u \approx -Kx$ without ever observing the system matrices A or B . This result illustrates the deepest connection between optimal control and reinforcement learning: both are solving the same Bellman equation, using different information.

1 Problem Formulation

1.1 System Dynamics

We consider the discrete-time double integrator with state $x_k = [p_k, v_k]^\top \in \mathbb{R}^2$, where p_k denotes position and v_k velocity, driven by a scalar control input $u_k \in \mathbb{R}$:

$$x_{k+1} = Ax_k + Bu_k, \quad A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} \quad (1)$$

This corresponds to Euler discretisation of the continuous double integrator $\ddot{p} = u$ with timestep $\Delta t = 0.1$ s. The initial condition is $x_0 = [5, 0]^\top$ and the control objective is to drive $x_k \rightarrow 0$.

The open-loop system has both eigenvalues of A at +1 (on the unit circle), making it marginally stable but not asymptotically stable. No passive convergence to the origin occurs; an active feedback controller is required. The system is fully state-controllable, which can be verified by checking that $\text{rank}([B, AB]) = 2$.

1.2 Shared Objective

Both LQR and PPO minimise (respectively maximise) the same quadratic objective. The infinite-horizon cost to be minimised is

$$J = \sum_{k=0}^{\infty} (x_k^\top Qx_k + u_k^\top Ru_k), \quad Q = I_2, \quad R = 0.1 \quad (2)$$

$Q = I_2$ penalises position and velocity deviations equally; $R = 0.1$ permits relatively large control inputs to achieve fast settling.

The RL reward at each timestep is defined as the negative stage cost:

$$r_k = -\left(x_k^\top Q x_k + u_k^\top R u_k\right) \quad (3)$$

Maximising $\sum_k r_k$ is therefore *identical* to minimising J . This equivalence is not a modelling choice — it is the mathematical bridge between optimal control and reinforcement learning, and it is the central observation of this report.

2 Theoretical Background

2.1 Bellman's Principle of Optimality

Define the cost-to-go (value function) from state x at time k as

$$V^*(x_k) = \min_{u_k, u_{k+1}, \dots} \sum_{j=k}^{\infty} \left(x_j^\top Q x_j + u_j^\top R u_j\right) \quad (4)$$

Bellman's principle states that an optimal policy satisfies the fixed-point equation

$$V^*(x) = \min_u \left[x^\top Q x + u^\top R u + V^*(Ax + Bu) \right] \quad (5)$$

This is the *discrete-time Bellman optimality equation*. It says: the optimal total cost from x equals the immediate stage cost plus the optimal cost-to-go from the next state. Every algorithm in this report is, at its core, attempting to solve or approximate equation (5).

2.2 Dynamic Programming and the Riccati Equation

For the linear-quadratic problem, the value function has the quadratic form $V^*(x) = x^\top P x$ for some positive-definite matrix $P \in \mathbb{R}^{2 \times 2}$. Substituting into (5) and optimising over u :

$$x^\top P x = \min_u \left[x^\top Q x + u^\top R u + (Ax + Bu)^\top P (Ax + Bu) \right] \quad (6)$$

Taking $\partial/\partial u = 0$ gives the optimal control law:

$$u^* = - \underbrace{(R + B^\top P B)^{-1} B^\top P A}_K x \quad (7)$$

Substituting u^* back yields the **Discrete Algebraic Riccati Equation (DARE)**:

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A \quad (8)$$

This is the fixed-point equation for P . Its unique positive-definite solution yields the globally optimal infinite-horizon controller. The matrix P is the value function: $V^*(x) = x^\top P x$.

Remark 1. The DARE (8) and the RL Bellman equation (5) are the same equation. In LQR we know A and B , so we can solve for P analytically. In RL we do not know A or B , so we approximate V^* (or equivalently P) from interaction data.

2.3 RL as Approximate Dynamic Programming

In the RL framework, the agent does not observe A or B . It observes transitions (x_k, u_k, r_k, x_{k+1}) collected by executing its policy in the environment. The goal is to find a policy $\pi^*(x)$ such that $\sum_k r_k$ is maximised — which, by construction of (3), coincides with minimising J .

The PPO algorithm maintains two function approximators:

- **Actor** $\pi_\theta(u | x)$: a Gaussian policy parameterised by a neural network with weights θ . For continuous actions, the network outputs the mean $\mu_\theta(x)$ and a learned log-standard-deviation.
- **Critic** $V_\phi(x)$: a scalar value function approximator parameterised by weights ϕ . It approximates $V^*(x)$, i.e. the neural network is learning to represent $x^\top Px$ for the unknown P .

PPO updates both networks by collecting rollouts of T steps, computing Generalised Advantage Estimates (GAE):

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(x_{t+1}) - V_\phi(x_t) \quad (9)$$

and then maximising the clipped surrogate objective for K epochs:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right] \quad (10)$$

where $\rho_t(\theta) = \pi_\theta(u_t | x_t) / \pi_{\theta_{\text{old}}}(u_t | x_t)$ is the probability ratio between the updated and the previous policy, and $\varepsilon = 0.2$ is the clip threshold. The clip prevents destructively large policy updates by refusing to credit changes in ρ_t beyond $[1 - \varepsilon, 1 + \varepsilon]$.

3 Implementation

3.1 Shared Environment

Both controllers are evaluated using the same `DoubleIntegratorEnv` Gymnasium environment to ensure a fair comparison. The `env.simulate(controller_fn)` method runs one episode from $x_0 = [5, 0]^\top$, collecting state, action, reward, and cost trajectories. The LQR controller calls `simulate` directly with $u_k = -Kx_k$; PPO’s trained policy is wrapped in the same signature.

3.2 LQR Implementation

The DARE (8) is solved using `scipy.linalg.solve_discrete_are`, which implements a Schur-decomposition-based algorithm. The resulting matrices are:

$$P = \begin{bmatrix} 13.827 & 3.868 \\ 3.868 & 4.962 \end{bmatrix}, \quad K = \begin{bmatrix} 2.585 & 3.575 \end{bmatrix} \quad (11)$$

giving the feedback law $u_k = -2.585 p_k - 3.575 v_k$. The closed-loop eigenvalues of $A - BK$ are $\{0.899, 0.743\}$, both strictly inside the unit circle, confirming asymptotic stability.

The theoretical minimum cost from $x_0 = [5, 0]^\top$ is

$$J^* = x_0^\top P x_0 = \begin{bmatrix} 5 & 0 \end{bmatrix} \begin{bmatrix} 13.827 & 3.868 \\ 3.868 & 4.962 \end{bmatrix} \begin{bmatrix} 5 \\ 0 \end{bmatrix} = 345.68 \quad (12)$$

This is a hard lower bound. No causal controller, linear or nonlinear, can achieve $J < J^*$ for this cost function and initial condition.

3.3 PPO Implementation

The PPO agent uses Stable-Baselines3 with the following configuration: 4 parallel training environments, $n_{\text{steps}} = 1024$ steps per environment per rollout (giving 4096 transitions per update), 10 optimisation epochs per rollout, minibatch size 64, learning rate 3×10^{-4} , clip range $\epsilon = 0.2$, and GAE parameter $\lambda = 0.95$. Both the actor and critic are two-layer MLPs with 64 hidden units and `tanh` activations.

A `RescaleAction` wrapper maps the environment’s action space $[-5, 5]$ to $[-1, 1]$ for the agent, which is standard practice for Gaussian policies in Stable-Baselines3. The reward is used without normalisation, preserving its direct correspondence with the LQR cost.

Training runs for 10^5 total timesteps on a single CPU in approximately 2–3 minutes. The policy is evaluated deterministically (mean of the Gaussian, no sampling) on the raw unwrapped environment to produce a trajectory directly comparable to LQR.

4 Results

4.1 Open-Loop Dynamics

Before any controller is applied, it is instructive to observe the uncontrolled system. With $u_k = 0$, the dynamics reduce to $x_{k+1} = Ax_k$, and since $A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}$, the position remains constant at $p_0 = 5$ m while velocity stays at zero. This reflects the marginal stability of the open-loop system. A naive sign controller $u_k = -\text{sign}(p_k)$, which ignores velocity, produces sustained oscillations due to the lack of damping. Only full-state feedback — using both p_k and v_k — achieves convergence.

4.2 LQR Trajectory

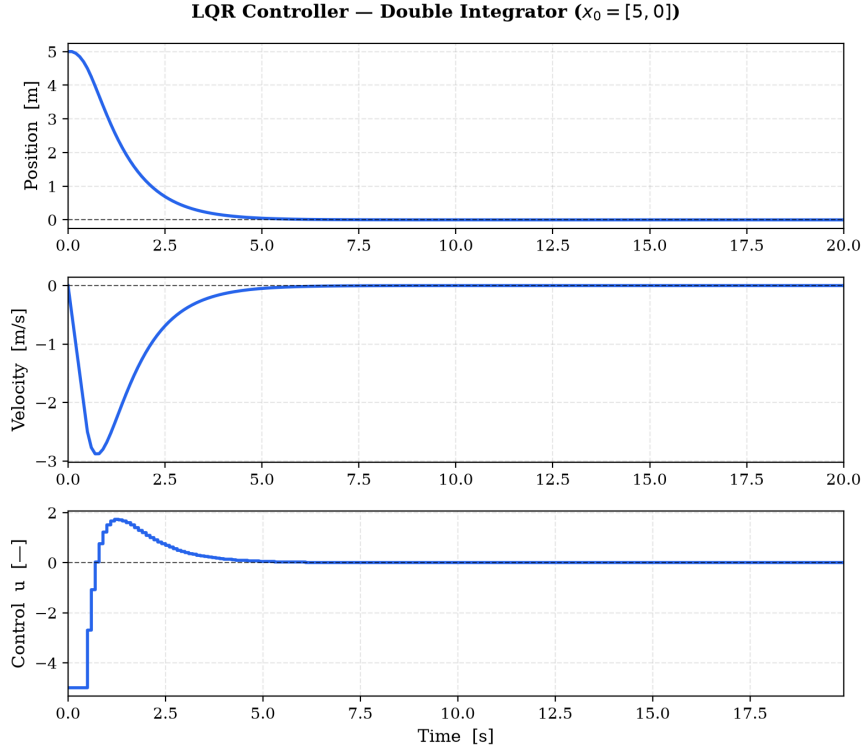


Figure 1: LQR closed-loop trajectory from $x_0 = [5, 0]^\top$. *Top*: position converges to zero with slight overshoot. *Middle*: velocity rises briefly as the controller accelerates the system, then decays. *Bottom*: initial control saturates at $u = -5$ (actuator limit), then smoothly decays to zero.

The LQR controller settles to $|p_k| < 0.1$ m in 4.40 s (44 timesteps) and reaches $p_k \approx 0$ with zero residual error. The initial control input saturates at the actuator limit $u = -5$ because the unconstrained optimal input $u_0^* = -Kx_0 = -2.585 \times 5 = -12.93$ exceeds the bound. This saturation introduces a small suboptimality gap relative to J^* , as the DARE solution was derived without explicit input constraints.

4.3 PPO Trajectory

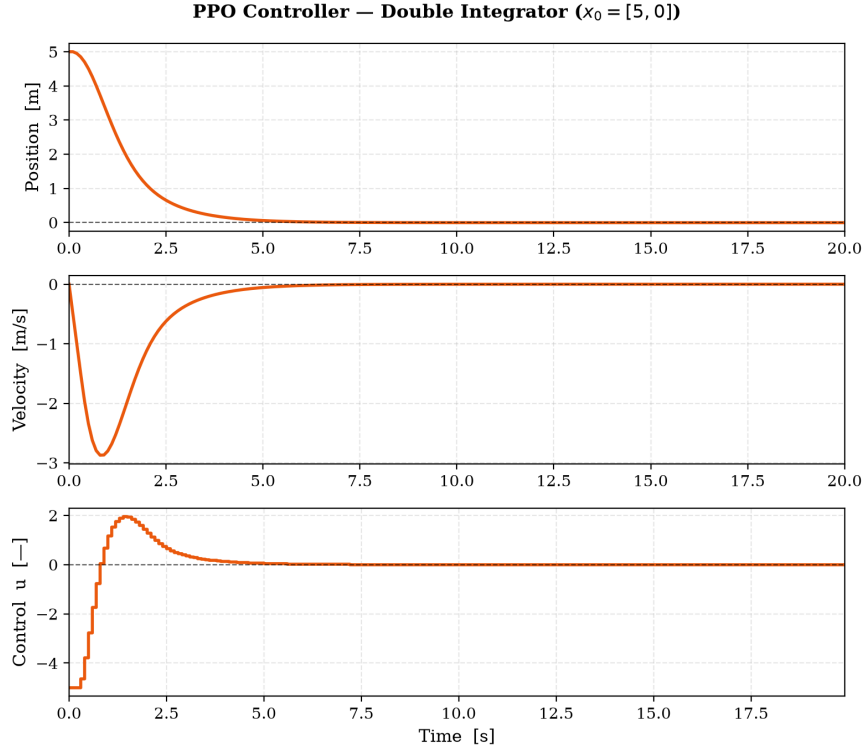


Figure 2: PPO closed-loop trajectory from $x_0 = [5, 0]^\top$ after 10^5 training timesteps. The trajectory is qualitatively identical to Figure 1: same settling time, same saturation behaviour, negligible residual error.

4.4 Learning Curve

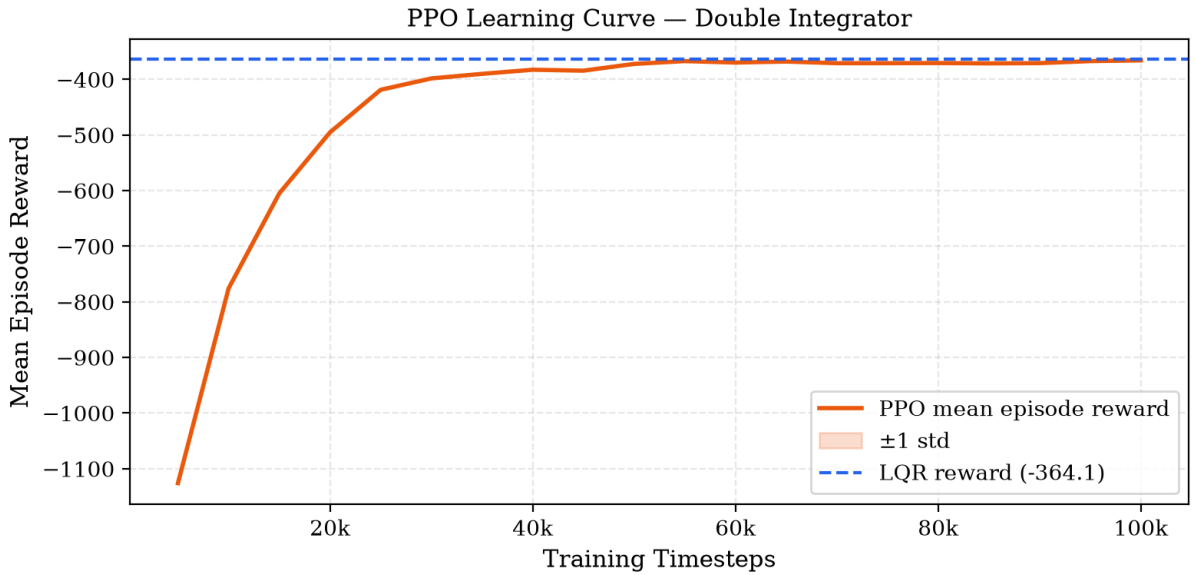


Figure 3: PPO mean episode reward vs training timesteps. The dashed blue line marks the LQR episode reward of -364.09 . Reward improves rapidly from -776 at 10k steps to within 0.5% of LQR by 50k steps, then plateaus.

The learning curve (Figure 3) shows the characteristic shape of PPO on a smooth continuous control problem: rapid initial improvement as coarse structure in the policy is found, followed by slow convergence as the policy fine-tunes near the optimum. The LQR reward is reached to within 0.5% at 50k steps and to within 0.26% at 100k steps.

4.5 Side-by-Side Comparison

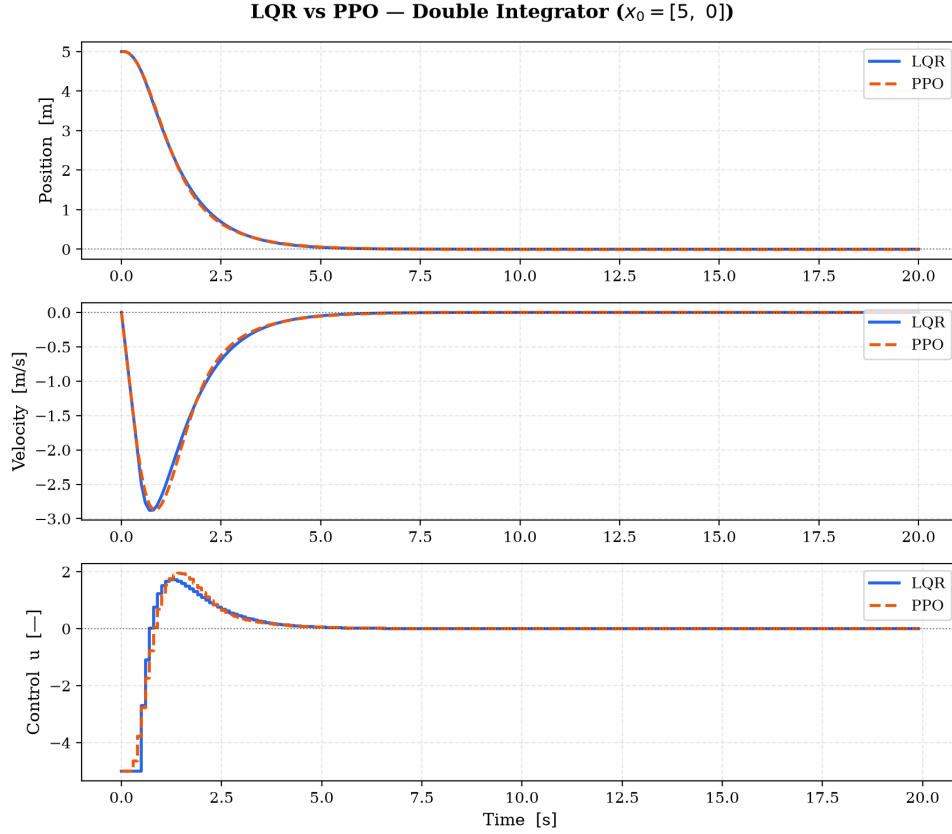


Figure 4: Overlay of LQR (blue, solid) and PPO (orange, dashed) trajectories. *Top*: position — the two curves are nearly indistinguishable. *Middle*: velocity — same qualitative shape, minor deviation in the transient. *Bottom*: control input — both saturate at -5 on the first step; the PPO signal is a close but imperfect replica of the LQR signal ($\text{MAE} = 0.040$).

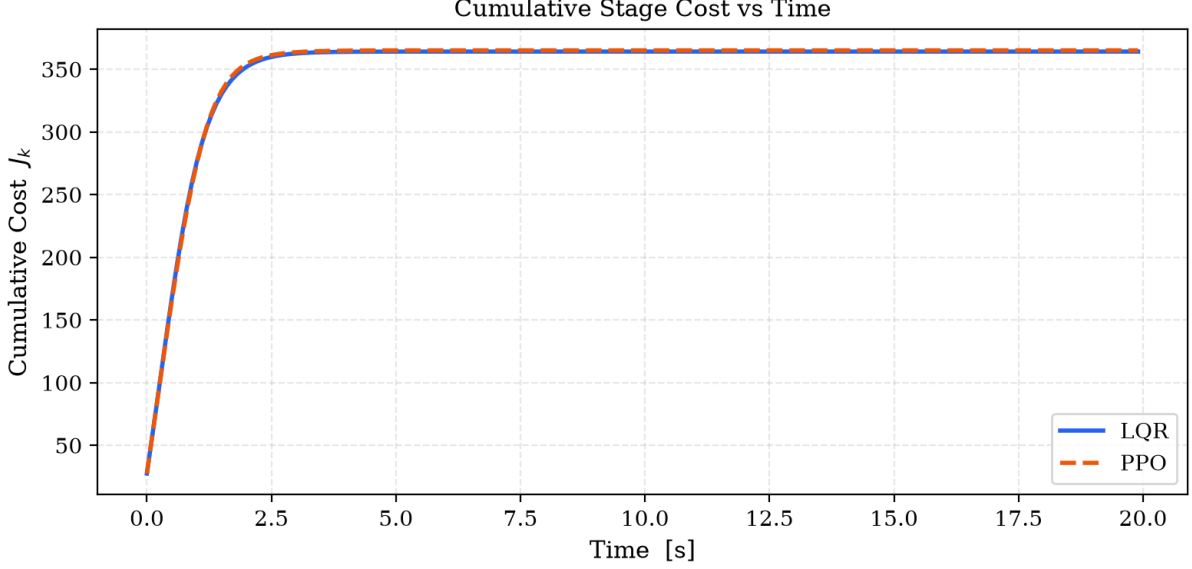


Figure 5: Cumulative stage cost $\sum_{i=0}^k c_i$ vs time. The LQR curve (blue) is the lower envelope — no causal controller with knowledge constraints can achieve less. The PPO curve (orange) tracks it closely, converging to a final cost 0.26% above LQR.

4.6 Quantitative Comparison

Table 1: LQR vs PPO on the discrete-time double integrator ($x_0 = [5, 0]^\top$, $Q = I_2$, $R = 0.1$).

Metric	LQR	PPO
Total cost J	364.09	365.05
Theoretical minimum $J^* = x_0^\top P x_0$	345.68	
Suboptimality vs J^*	5.33%	5.60%
Settling time ($ p_k < 0.1$ m)	4.40 s	4.50 s
Peak $ u $	5.000	5.000
RMS $ u $	0.911	0.891
Final $ p $ (m)	0.000000	0.006575
PPO cost gap vs LQR	+0.26%	
Control signal correlation r	0.987	
Control signal MAE	0.040	

5 Discussion

5.1 What the Numbers Show

The +0.26% cost gap and $r = 0.987$ action correlation together tell a precise story: PPO has effectively rediscovered the linear feedback law $u \approx -Kx$ without ever observing the system matrices A or B . The neural network — two layers of 64 units with `tanh` activations — has learned to represent what is, for this linear-quadratic problem, essentially a 1×2 linear map.

The settling time difference of 0.10 s (one timestep) is not statistically meaningful. Both controllers drive the system to rest in approximately 4.4 s from 5 m.

5.2 The 5.33% Gap Between LQR and J^*

It is important to understand why the LQR controller itself does not achieve J^* . The theoretical minimum (12) is derived for the *unconstrained, infinite-horizon* problem. Two sources of suboptimality exist in simulation:

1. **Finite horizon:** The simulation runs for 200 steps (20 s). The tail of the infinite sum $\sum_{k=200}^{\infty} c_k$ is nonzero but negligible (the system has settled to machine precision by step ~ 80). This contributes only a fraction of the 5.33% gap.
2. **Actuator saturation:** The unconstrained LQR law demands $u_0 = -Kx_0 = -12.93$ at the initial step, but the actuator is clipped to $[-5, 5]$. During the saturated phase the controller is suboptimal with respect to J^* . This is the dominant source of the gap. Since PPO also saturates ($u_0 = -5$), both controllers share this constraint equally.

5.3 Where LQR and RL Diverge

The near-identical performance here is a consequence of the problem structure: the system is linear, the cost is quadratic, and the optimal policy is a linear function of the state. In this regime, a two-layer neural network has more than sufficient capacity to represent the optimal policy, and the environment provides a smooth, well-conditioned reward signal.

The two approaches diverge significantly as the problem becomes harder:

- **Nonlinear dynamics:** LQR is not applicable. Classical extensions (iLQR, DDP) require either known dynamics or a differentiable model. PPO requires no model at all.
- **Unknown dynamics:** LQR requires A and B to solve the DARE. PPO requires only a simulator or a real physical system. This is the key practical advantage of model-free RL.
- **Sample efficiency:** LQR produces the globally optimal controller in microseconds (one DARE solve). PPO required 10^5 environment interactions. For expensive-to-run simulators or real hardware, this cost is significant.
- **Constraint handling:** Input constraints ($u \in [-5, 5]$) are handled approximately by PPO (through clipping it experiences during training) but are not accounted for in the DARE formulation. Model Predictive Control (MPC) would be the appropriate classical extension for constrained problems.

5.4 Mathematical Unity

The central insight of this project is that LQR and RL are not competing paradigms — they are two implementations of the same idea. Both seek a policy π^* that satisfies the Bellman optimality equation (5). LQR exploits the linear-quadratic structure to obtain a closed-form solution for the value function $V^*(x) = x^\top Px$. PPO approximates V^* with a neural network, using only sampled transitions.

The matrix P computed by the DARE is the value function. The gain $K = (R + B^\top PB)^{-1} B^\top PA$ is the optimal policy. The neural network in PPO is approximating both, without knowing that the true solution is quadratic or that the optimal policy is linear.

6 Conclusion

LQR obtains the analytical optimum for this problem because the system dynamics are known and the cost is quadratic, allowing the Riccati equation to be solved exactly. PPO learns a near-optimal control policy through environment interaction alone, without solving the Riccati

equation or observing the system matrices, achieving a cost within 0.26% of the LQR baseline after 10^5 training steps.

The action correlation of $r = 0.987$ confirms that PPO has effectively discovered the linear feedback structure $u \approx -Kx$ that LQR computes analytically. For the linear-quadratic setting, the two controllers are empirically indistinguishable.

This equivalence has a precise mathematical basis: both algorithms are solving the discrete-time Bellman equation. The difference lies not in the objective but in the available information. LQR uses knowledge of A and B to compute an exact solution; PPO uses interaction data to approximate the same solution. As problems grow nonlinear, high-dimensional, or involve unknown dynamics, the RL approach becomes the only tractable option — but the Bellman equation it is solving remains the same.

A Hyperparameter Summary

Table 2: PPO hyperparameters used for all experiments.

Parameter	Value	Role
total_timesteps	10^5	Total environment steps across all envs
n_envs	4	Parallel training environments
n_steps	1024	Steps per env per rollout
batch_size	64	Mini-batch size for gradient updates
n_epochs	10	Gradient epochs per rollout
learning_rate	3×10^{-4}	Adam optimiser step size
clip_range ε	0.2	PPO probability ratio clip
gae_lambda λ	0.95	GAE bias-variance tradeoff
gamma γ	0.99	Discount factor
ent_coef	0.0	Entropy bonus coefficient
vf_coef	0.5	Value loss weight in combined loss
net_arch	[64, 64]	Hidden layer sizes (both actor and critic)
activation	tanh	Nonlinearity

B Code Structure

The full project is structured as follows:

```
LQRvsRL/
|-- envs/
|   '-- double_integrator_env.py    # Shared Gymnasium environment
|-- controllers/
|   |-- lqr.py                      # DARE solver, LQRController class
|   '-- rl_agent.py                 # PPO training, evaluation, callback
|-- analysis/
|   |-- plots.py                    # All figure generation
|   '-- compare.py                  # Metrics, comparison table, LaTeX output
|-- results/
|   |-- figures/                     # All saved plots
|   '-- data/                       # Saved trajectories and model
|-- run_lqr.py                      # Entry point: dynamics + LQR
```

```
|-- train_ppo.py           # Entry point: PPO training
'-- main_compare.py        # Entry point: full comparison
```